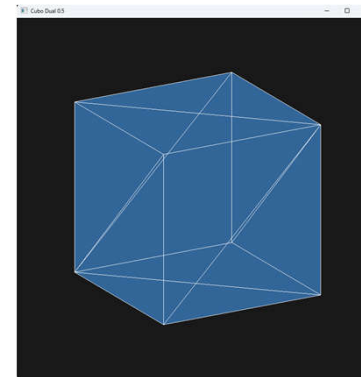
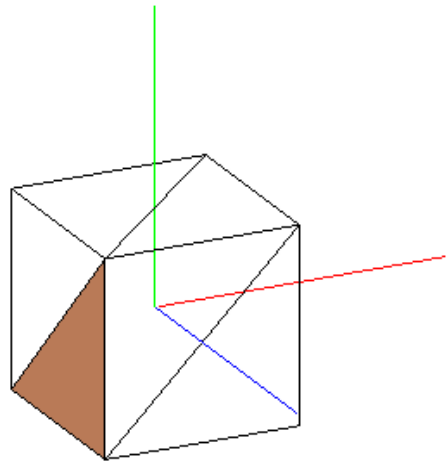


Modelado de objetos 3D



Temario

Tema 1: Introducción: Revisión histórica da computación gráfica

Tema 2: Estándares gráficos Evolución histórica, OpenGL vs DirectX

Tema 3: Formas 2D Tema 4: Transformaciones geométricas, 2D, 3D,

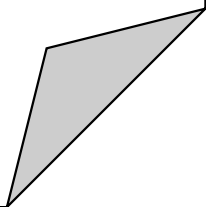
Tema 5: Proyección, modelo de cámara sintética.

Tema 6: Modelado e texturas

Tema 7: Color, Iluminación y sombreado

Tema 8: Determinación de superficies visibles, z-Buffer

Tema 9: Shaders GLSL



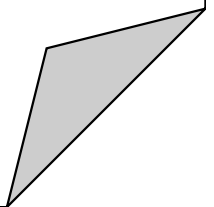
Temario: Tema 6: Modelado y texturas

Tema 6: Modelado y texturas.

6.1. Diseño avanzado de objetos

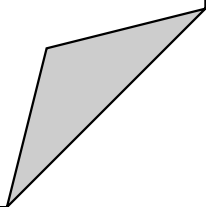
6.2. Librerías auxiliares

6.3. Texturización.

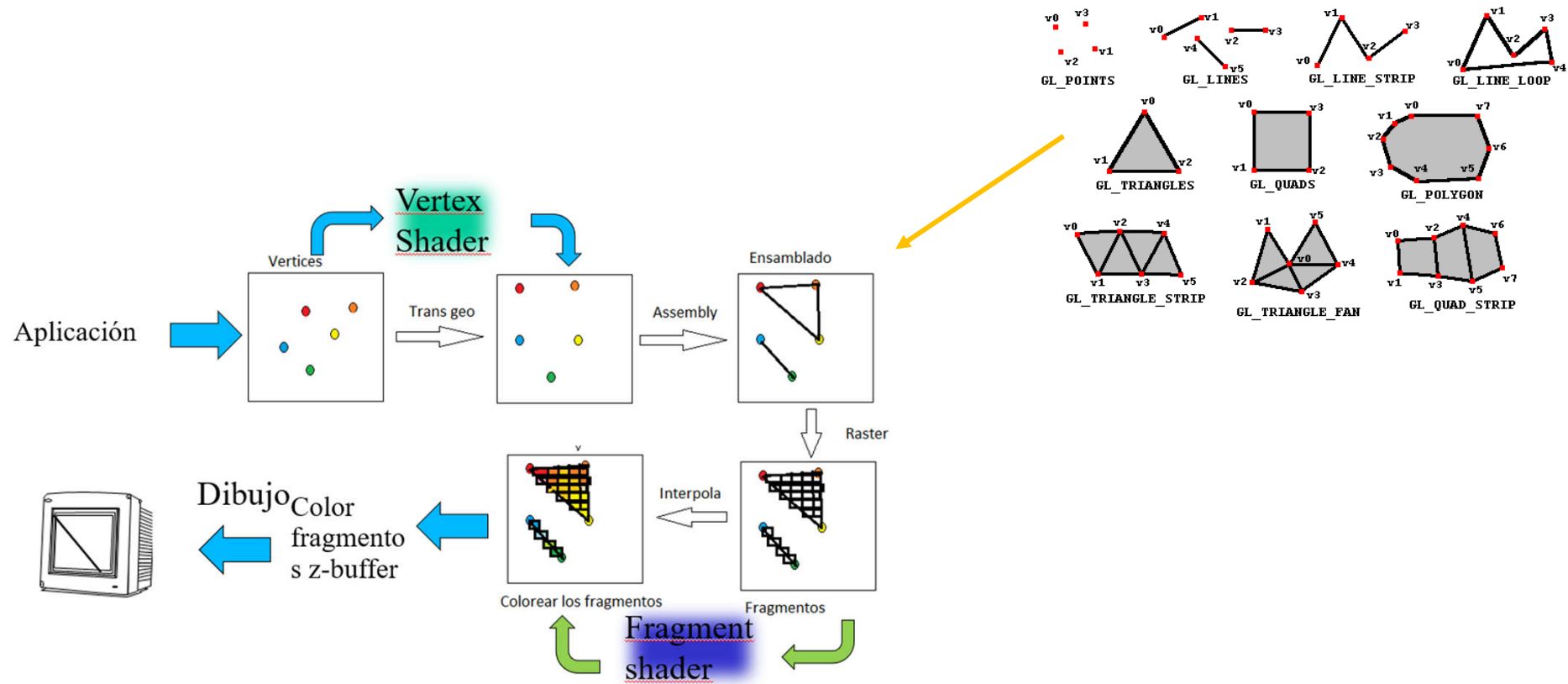


Bibliografía

- GRÁFICOS POR COMPUTADORA CON OPENGL, Hearn, Donald ; Baker, Pauline.
 - Pdf. Capítulo 2, pp 35-47
- Real-Time Rendering, Third Edition: Edition 3. Tomas Akenine-Möller Eric Haines Naty Hoffman.
- Redbook version 8.0 o superior.
- Learnopengl.com

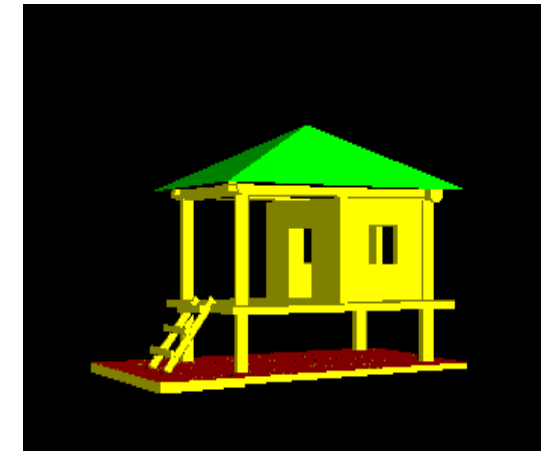
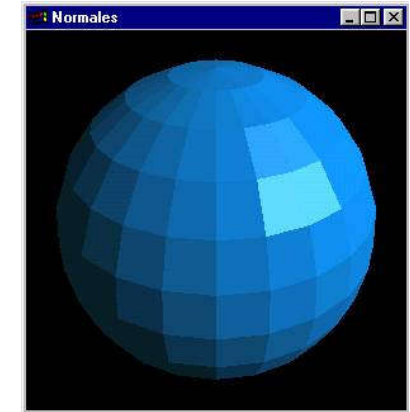


PIPELINE



Objetos sólidos

- ¿Qué es modelar un objeto?
 - Dar forma a un objeto.
 - Modelado interior o exterior.
- **Representación de superficie.** proporciona información sobre:
 - La superficie. Debe dar cuenta de su geometría.
 - Propiedades visuales cómo se comporta frente a la luz.
 - Color.
 - Textura.
 - Alguna propiedad física (como la elasticidad) si se va a efectuar una simulación física sobre el objeto.

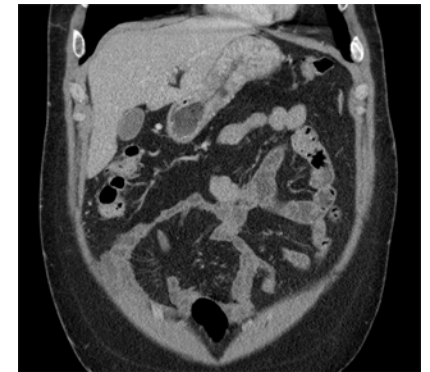
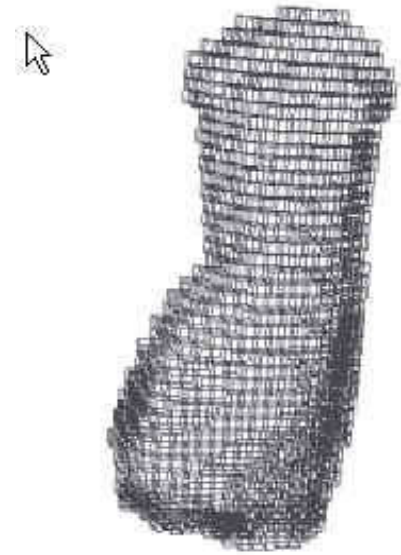


Objetos Sólidos-interior

Partición espacial:

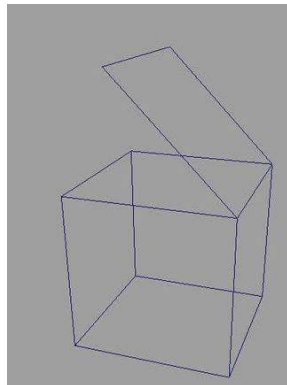
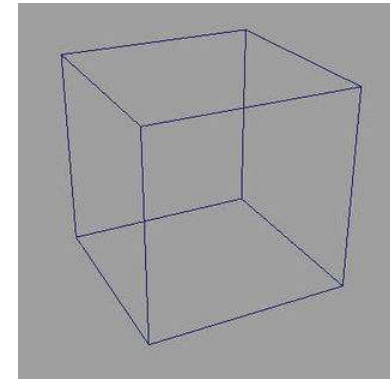
La representación volumétrica de un objeto se logra dividiendo su interior en pequeñas regiones sólidas que no se superponen entre sí. Estas regiones pueden adoptar diferentes formas geométricas, pero comúnmente se utilizan cubos o vóxeles (volumetric pixels). Este enfoque es esencial para representar datos tridimensionales de manera eficiente y se emplea ampliamente en áreas como gráficos por computadora, modelado 3D, simulaciones físicas y visualización científica.

Una técnica particularmente conocida en este contexto es el algoritmo **Marching Cubes**, que se utiliza para generar superficies tridimensionales a partir de datos volumétricos. Este algoritmo divide el espacio en cubos y evalúa los valores de los vértices para determinar cómo conectar las aristas del cubo y formar polígonos, generalmente triángulos. Esta técnica es útil para representar superficies implícitas, como las obtenidas en imágenes médicas (tomografías computarizadas o resonancias magnéticas) o simulaciones físicas.

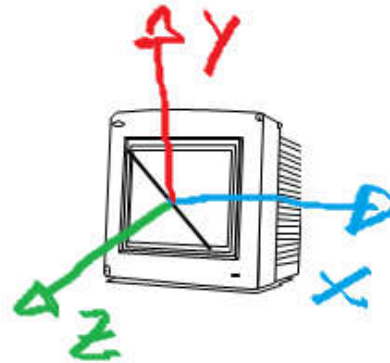
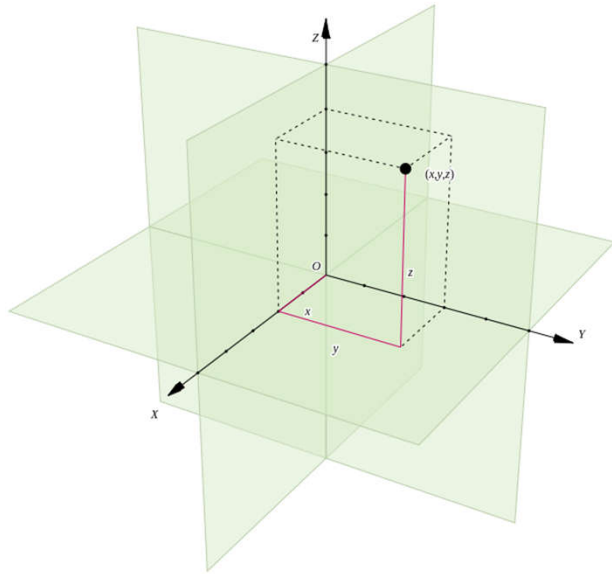


Objetos 3D, Representación de Superficies

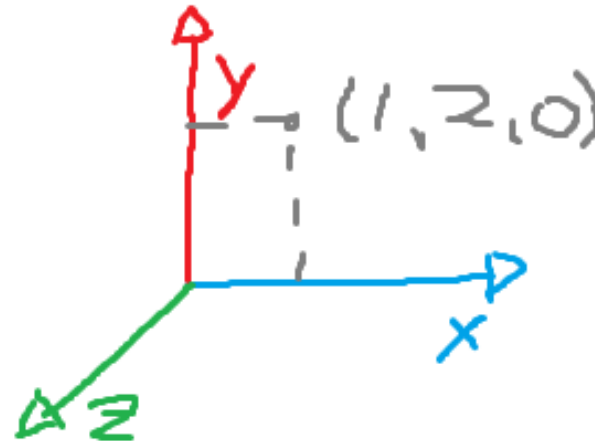
- Define el contorno del objeto sin hacer referencia a su volumen interior.
- La representación de objetos 3D.
 - Las superficies representadas deben ser cerradas.



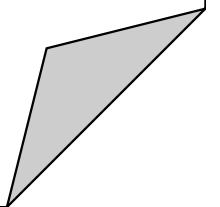
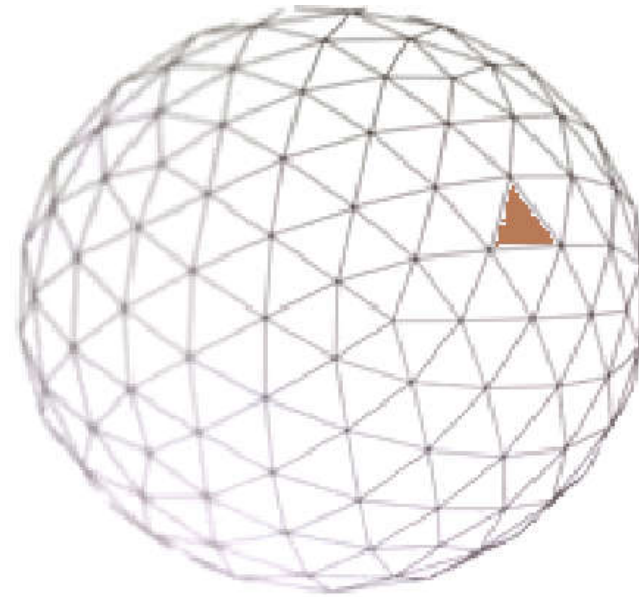
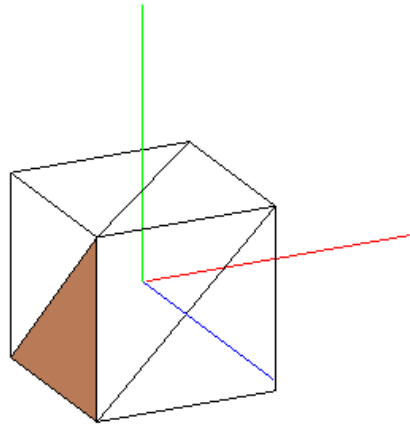
Ejes coordenados, Puntos, vectores.



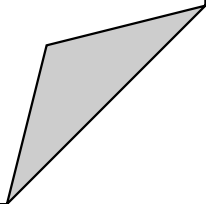
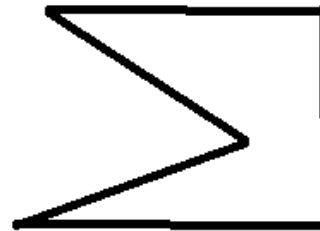
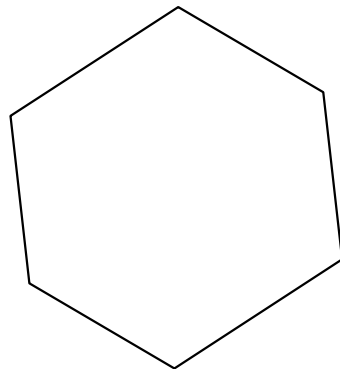
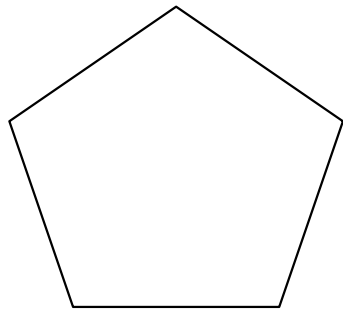
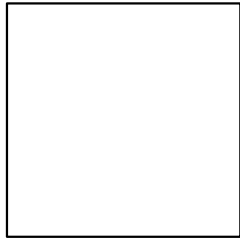
Por defecto
 $(-1,-1,-1)(1,1,1)$



Triangulación.-tessellation



Triangulación.-tessellation problema



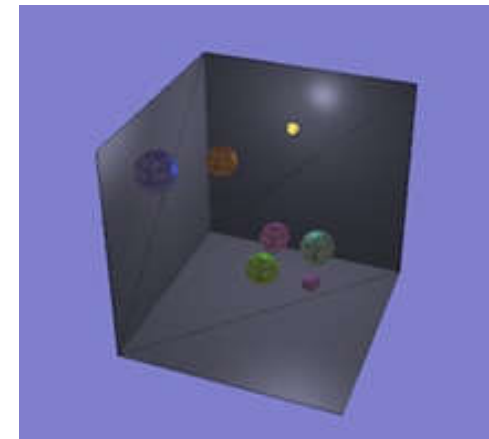
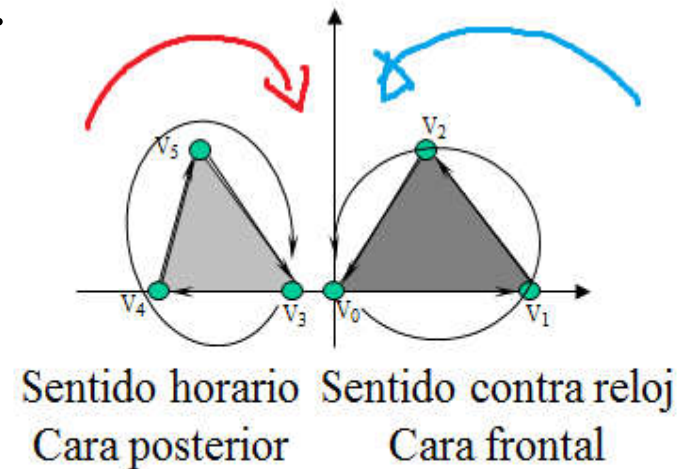
Identificación de caras

Es muy importante el orden en que se especifican los vértices ya que esto determina la normal de la cara.

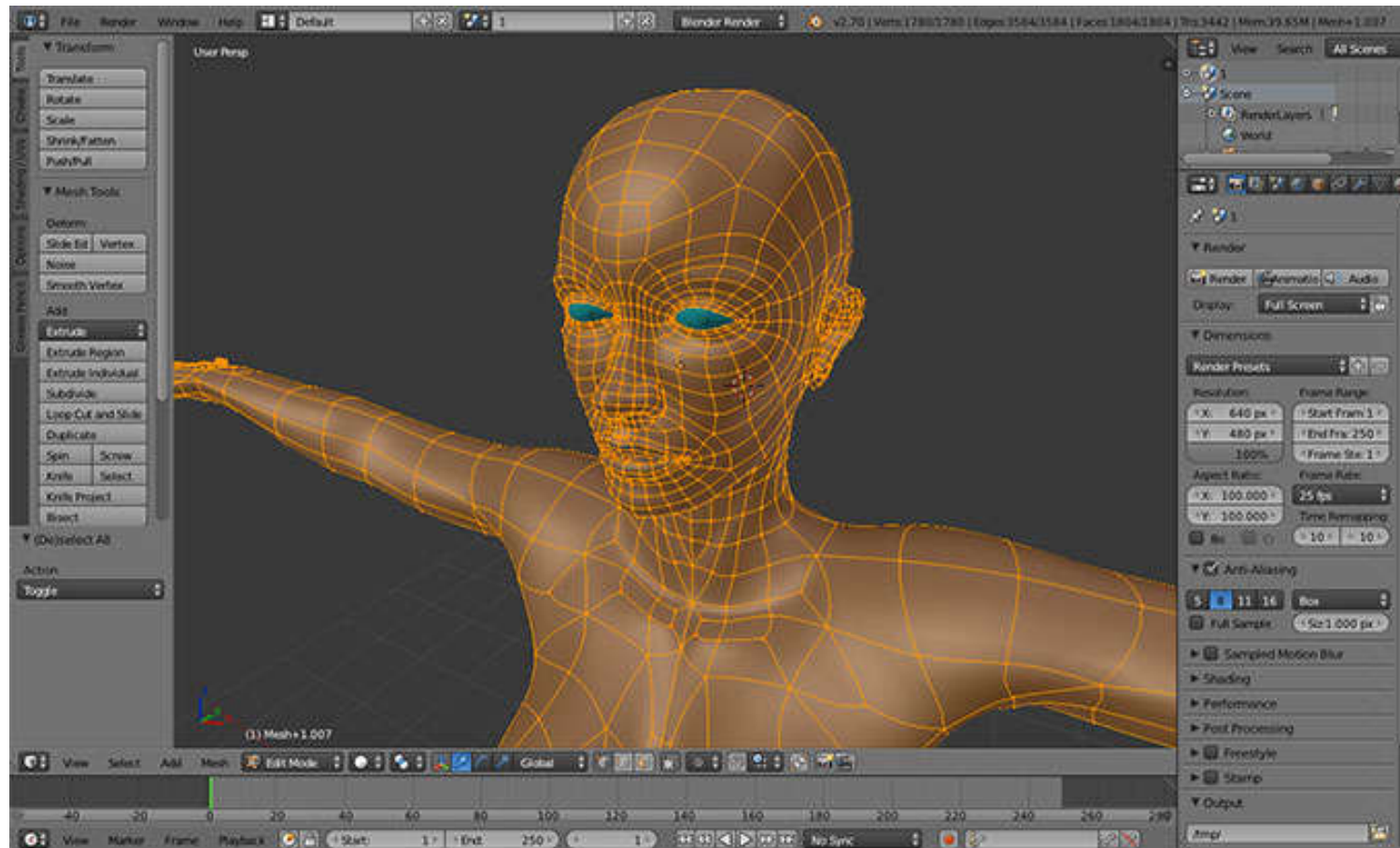
- Para un punto $P(x, y, z)$;
 $Ax + By + Cz + D = 0$; En la superficie.
 $Ax + By + Cz + D < 0$; está dentro;
 $Ax + By + Cz + D > 0$ esta fuera

```
glEnable (GL_CULL_FACE);  
glDisable (GL_CULL_FACE);  
glEnable(GL_NORMALIZE);
```

```
glFrontFace(GL_CW);  
glFrontFace(GL_CCW);
```

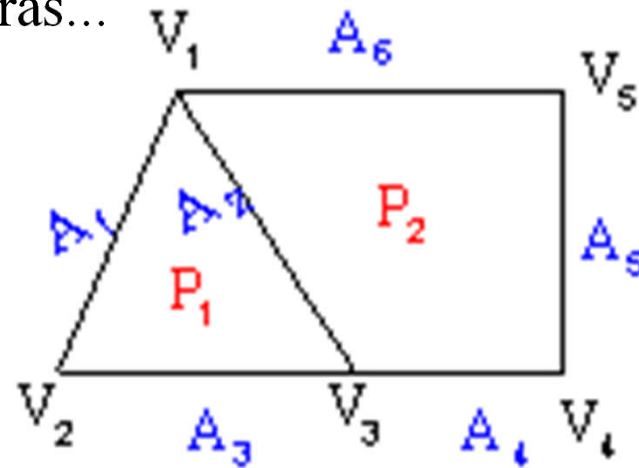


Crear modelos compejos.



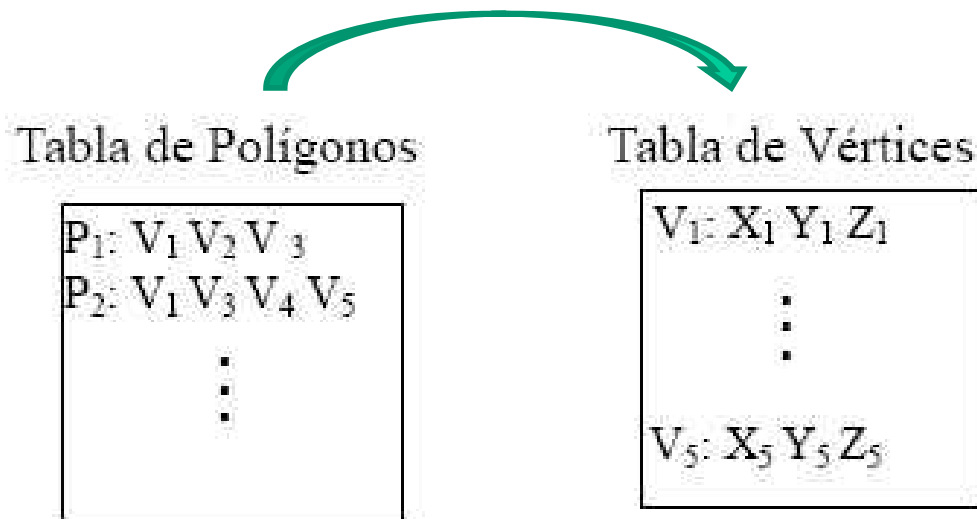
Representación superficie. MP. Tablas.

- Para la representación de un objeto en base a polígonos necesitamos conocer vértices y sus atributos y representarlos. Y almacenarlos de alguna forma.
- Tablas de dos tipos:
 - **Tablas geométricas** incorporan la información sobre los vértices y los parámetros que proporcionan información sobre los polígonos o la superficie que determinan.
 - **Tablas de atributos** informan sobre las propiedades físicas de los polígonos, color transparencia...texturas...
- Puede ser de tres tipos.
 - Sin indexar.
 - Indexada
 - Doblemente indexada



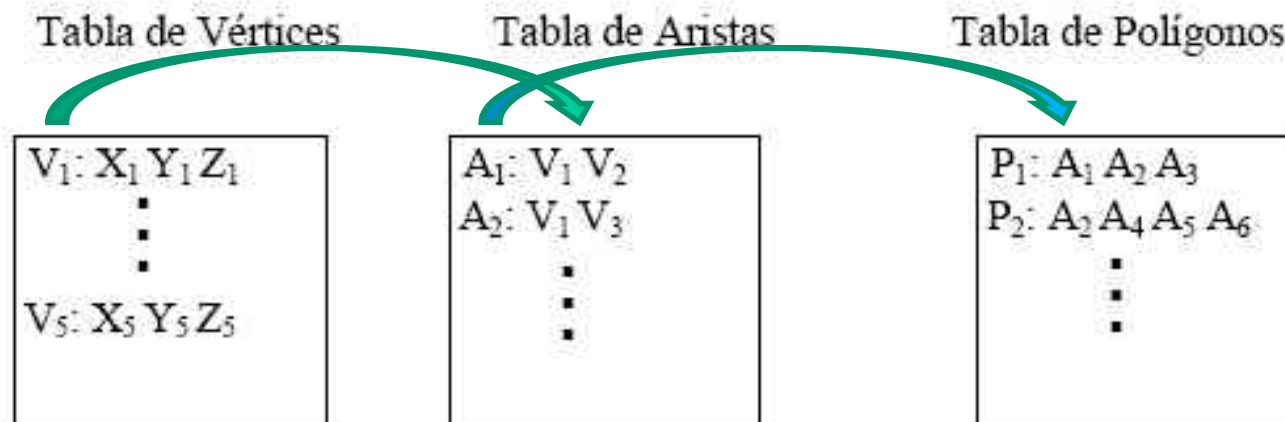
Tablas Indexadas

- Simplificar eliminando las aristas utilizando únicamente una lista de vértices y una de polígonos.
 - Para cada polígono los vértices que lo definen.
 - Esta representación es la empleada por el formato .obj de Aliaswavefron.



Tablas doblemente indexadas. Geometría

- Tabla de vértices: vértice, tres coordenadas
- Tabla de aristas, Para cada arista, tendríamos los dos vértices que la forman y, si se desea, los dos polígonos que la comparten.
- Tabla de Polígonos. Para cada polígono guardaríamos las aristas que lo forman.



.obj

```
# Wavefront OBJ file..\cubo.obj
# Bounding box of geometry = (-
0.5,-0.5,-0.5) to (0.5,0.5,0.5).
```

```
mtllib cubo.mtl
```

```
# Coordinates for object 'cubo'
```

```
v -0.500000 -0.500000 0.500000
v 0.500000 -0.500000 0.500000
v -0.500000 0.500000 0.500000
v 0.500000 0.500000 0.500000
v -0.500000 0.500000 -0.500000
v 0.500000 0.500000 -0.500000
v -0.500000 -0.500000 -0.500000
v 0.500000 -0.500000 -0.500000
# 8 vertices
```

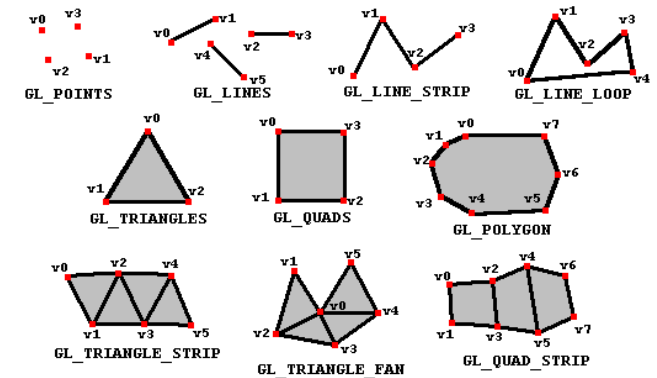
```
vt -1.000000 0.0 0.0
vt -1.000000 1.000000 0.0
vt 0.0 0.0 0.0
vt 0.0 1.000000 0.0
vt 0.0 2.000000 0.0
vt 0.0 3.000000 0.0
vt 0.0 4.000000 0.0
```

```
vt 1.000000 0.0 0.0
vt 1.000000 1.000000 0.0
vt 1.000000 2.000000 0.0
vt 1.000000 3.000000 0.0
vt 1.000000 4.000000 0.0
vt 2.000000 0.0 0.0
vt 2.000000 1.000000 0.0
# 14 texture vertices
```

```
usemtl default__2
```

```
s off
f 1/3 2/8 3/4
f 2/8 4/9 3/4
f 3/4 4/9 5/5
f 4/9 6/10 5/5
f 5/5 6/10 7/6
f 6/10 8/11 7/6
f 7/6 8/11 1/7
f 8/11 2/12 1/7
f 2/8 8/13 4/9
f 8/13 6/14 4/9
f 7/1 1/3 5/2
f 1/3 3/4 5/2
# 12 polygons
```

Las Primitivas, ensamblado



```
glDrawArrays(GL_LINES, 0, 8);
```

GL_POINTS: Los vértices pintan puntos individuales

GL_LINES: Los vértices se toman de dos en dos para formar segmentos

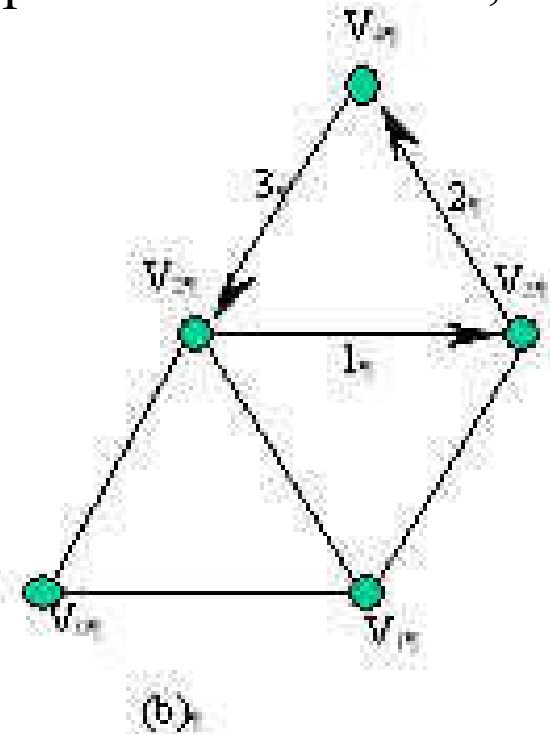
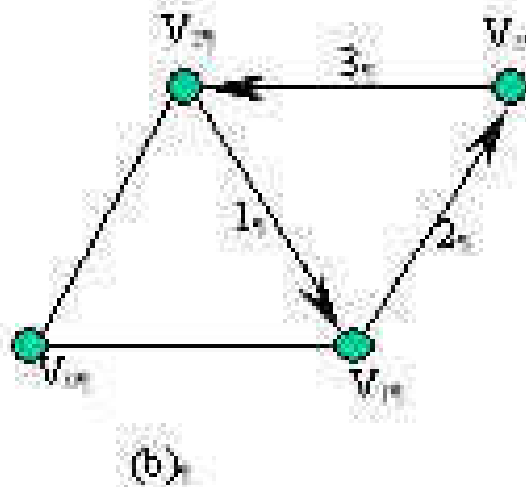
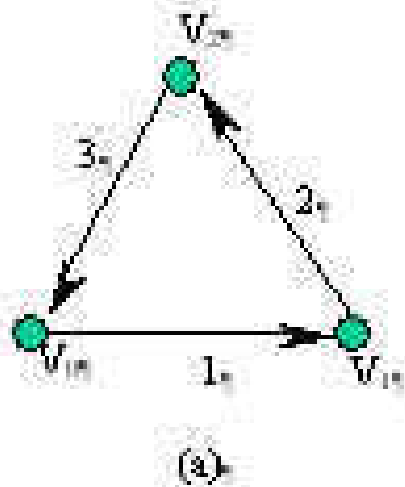
GL_LINE_STRIP: Los vértices se utilizan para definir una sucesión de segmentos.

GL_LINE_LOOP: Igual que el anterior, pero desde el último vértice se traza una línea hasta el primero de forma que se genera siempre una figura cerrada.

- **GL_TRIANGLES:** Los vértices especifican triángulos de 3 en 3.
- **GL_TRIANGLE_STRIP:** Los vértices van formando una tira de triángulos.
- **GL_TRIANGLE_FAN:** Los vértices forman un abanico de triángulos
- **GL_QUADS:** Los vértices se toman de cuatro en cuatro para formar cuadriláteros.
- **GL_QUAD_STRIP:** Los vértices especificados se usan para formar una tira de cuadriláteros.
- **GL_POLYGON:** Los vértices especificados se usan para construir un polígono convexo.

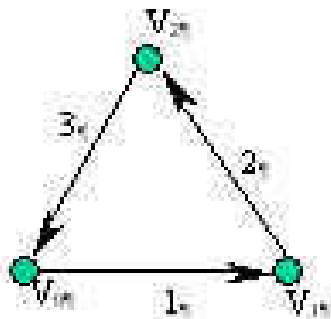
Tiras de triángulos, GL_TRIANGLE_STRIP

- El primer triángulo se forma con los tres primeros vértices en el orden en el que son dados.
- A partir de ahí, si el siguiente vértice es par los dos primeros vértices se invierten V_1, V_2 and $V_3 \rightarrow V_3, V_2$ and V_4 , si es par se mantienen V_3, V_2 and $V_4 \rightarrow$ of V_3, V_4 and V_5

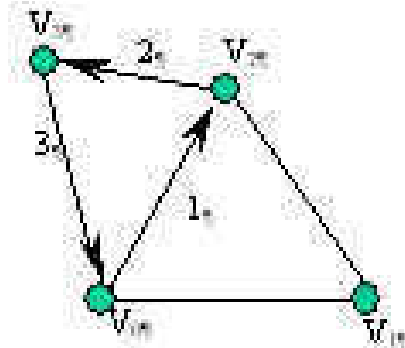


Abanicos de triángulos: GL_TRIANGLE_FAN

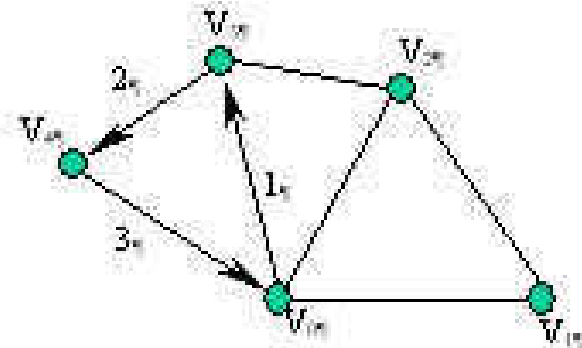
- El primer vértice dado en la lista
- El último vértice del triángulo anterior
- El siguiente vértice de la lista que aún no ha sido usado.



(a)



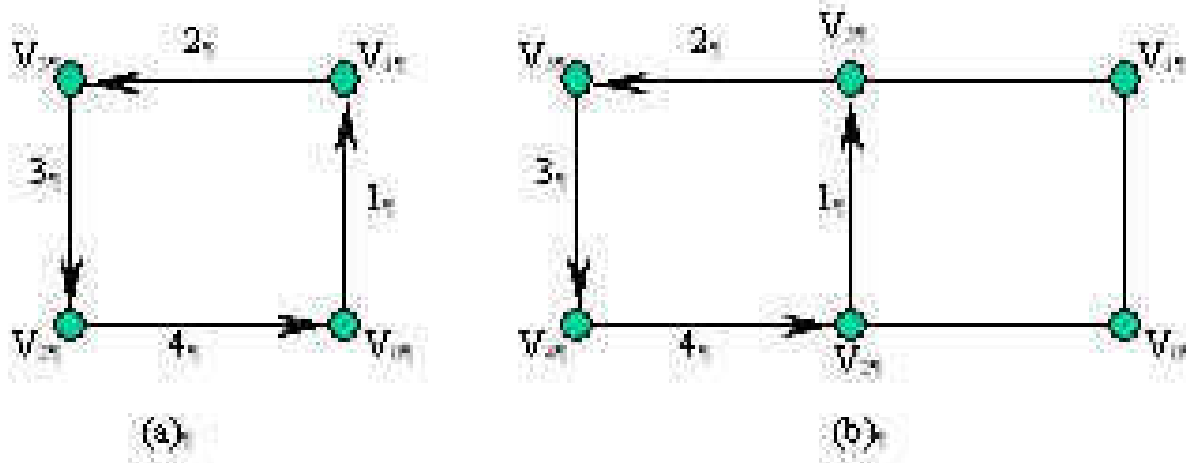
(b)



(c)

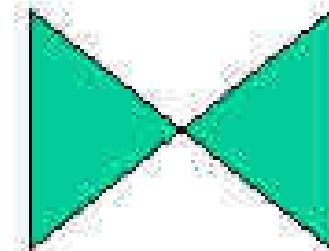
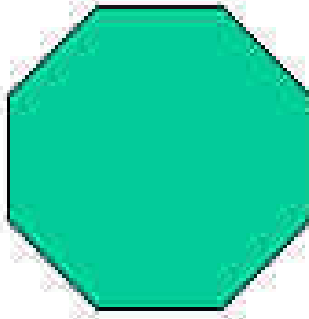
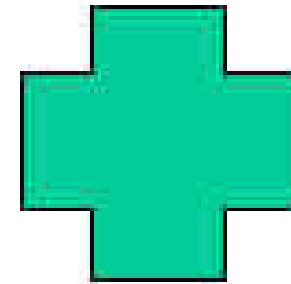
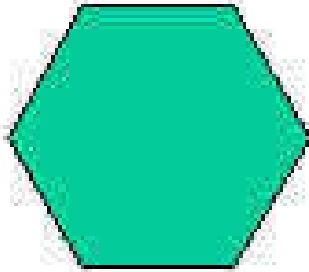
Tiras de cuadriláteros, GL_QUAD_STRIP

- Para formar el primer cuadro se utilizan los cuatro primeros vértices
- De los cuales el primer par de vértices se toma en el orden en el que se da mientras que el segundo par se toma en orden inverso.
- Para el resto de los cuadriláteros se toma el siguiente par en orden inverso y se utiliza el último par del cuadrilátero ya pintado en el orden dado

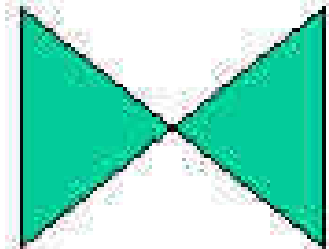
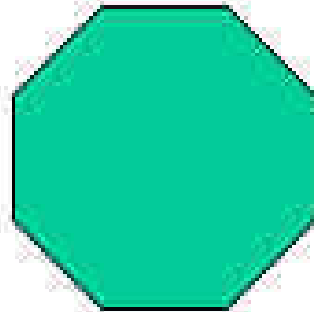
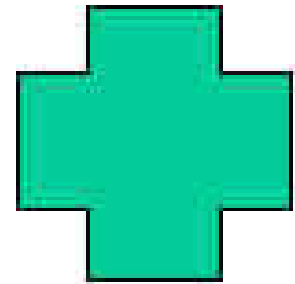
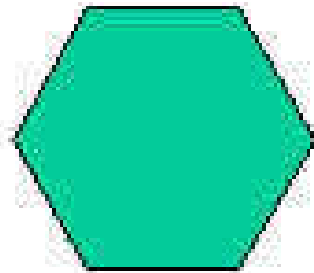


Polígonos, GL_POLYGON:

- Diseño de polígonos.
- Debemos tener en cuenta una serie de restricciones.
 - Los polígonos deben ser necesariamente convexos sin cavidades.
 - Lados no pueden cortarse.



Tutorial



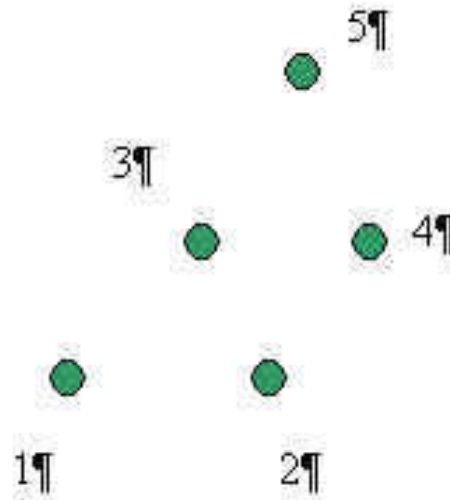
Problemas

Que figura genera cada secuencia de los vértices que se detalla a continuación. No se dibujan las caras traseras

`glPolygonMode(GL_FRONT, GL_FILL).`

`glBegin(GL_TRIANGLES);`

- 1) Secuencia, 1, 2, 3,4, 5.
- 2) Secuencia 3, 2, 1, 3, 4, 5.
- 3) Secuencia 2, 1, 3, 4, 3, 2, 3, 5, 4.



Problemas

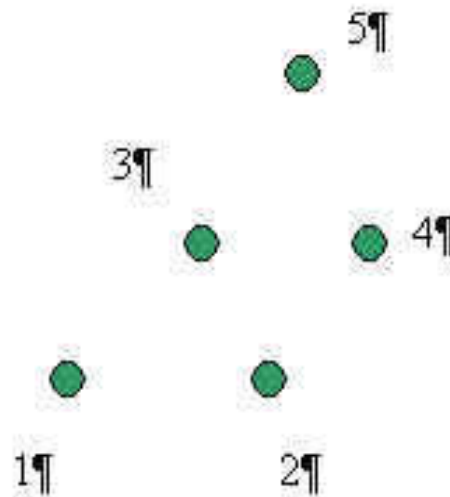
Dibuja lo que se vería para la secuencia. No se dibujan las caras traseras

`glPolygonMode(GL_FRONT, GL_FILL).`

`glBegin(GL_TRIANGLE_STRIP).`

Secuencia 2, 1, 3, 4, 5.

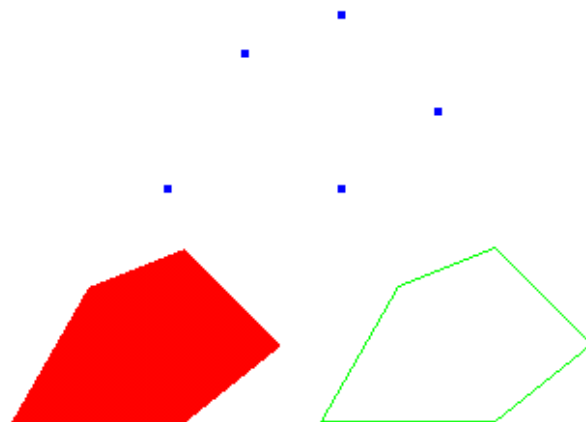
Secuencia, 1, 2, 3, 4, 5.



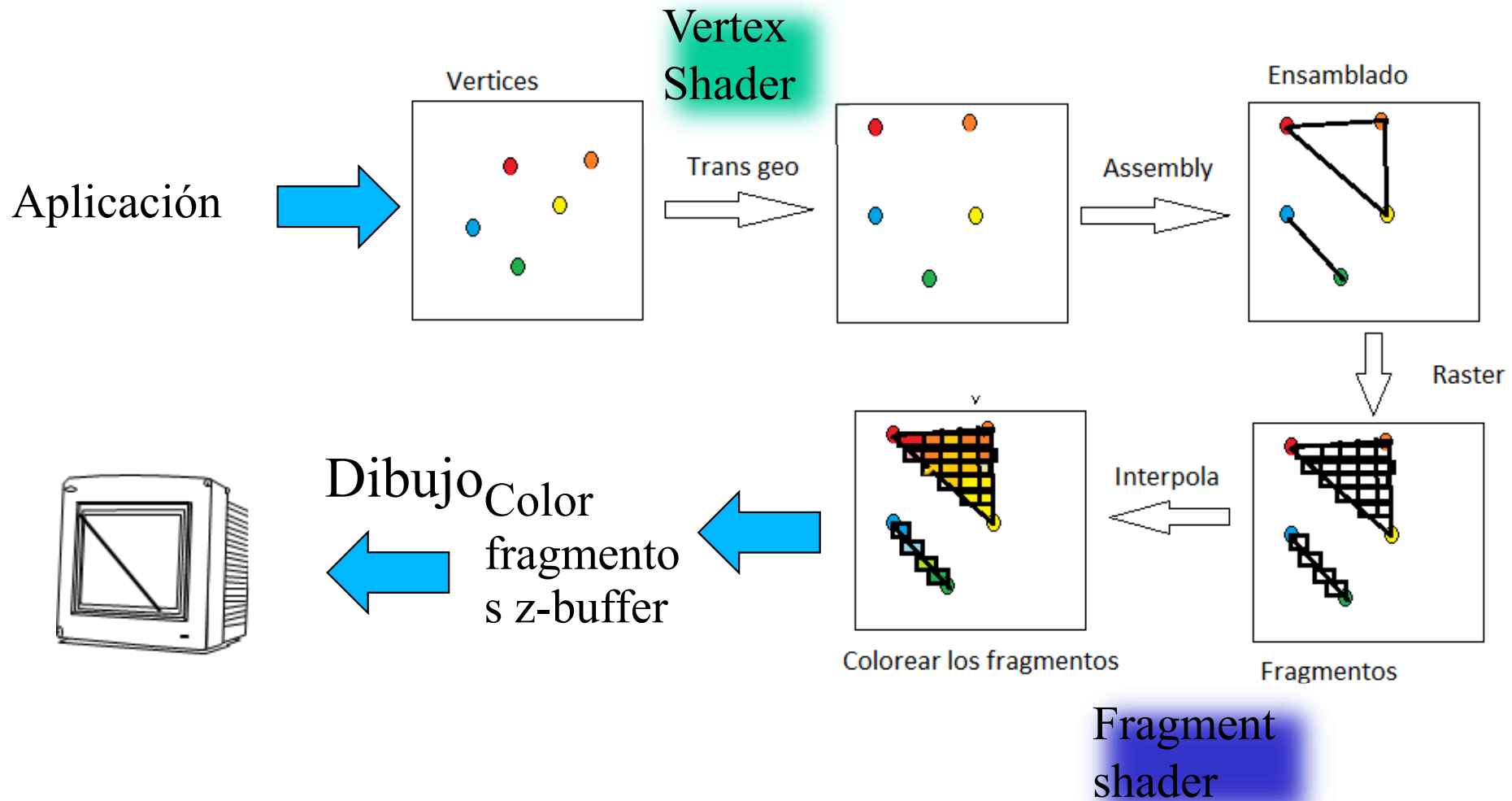
Modos poligonales. `glPolygonMode`

Especifica el modo de rasterizar los polígonos

- `glPolygonMode(GLenum cara, GLenum modo);`
 - `cara`: Especifica a que cara afectan los cambios, puede tomar los valores `GL_FRONT`, `GL_BACK` o `GL_FRONT_AND_BACK`.
 - `modo`: Especifica como queremos que se pinte la cara del polígono indicada, si rellena con líneas o con puntos para lo cual este parámetro toma los valores `GL_FILL`, `GL_LINE` o `GL_POINT`.



Dibujar los ejes con glfw y opengl 3.3



Tipo de Shaders

- **Vertex shaders.**

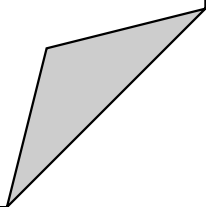
- Función que recibe como parámetro vértices.
- Manipula vértices en 3D; Color, coordenadas de textura, orientación y tamaño de vértices.
- Su propósito es transformar las coordenadas 3D de los vértices en otras coordenadas que ya veremos cuales son.
- La salida debe ser la posición de los vértices.

- **Fragment shaders (OpenGL)**

- Recibe la información de cada fragmento.
- Un fragmento es toda la información necesaria para dibujar un pixel.
- Se aplica en la última etapa de rasterizado.
- Su propósito es obtener el color de cada pixel.
- La salida es el color de cada pixel.

- **Que pasa después del Fragment Shader**

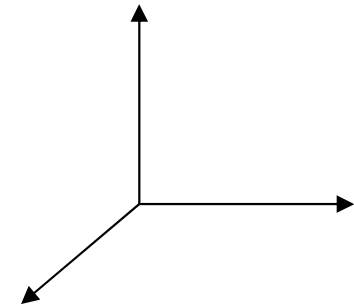
- Se realizan una serie de comprobaciones relativas al Alpha test, Blending test y Depth test.



Como mando los vértices

- Debo usar arrays de vértices de la forma.

```
float vertices[] = {
    0.0f, 0.0f, 0.0f, // x
    .5f, 0.0f, 0.0f,
    0.0f, 0.0f, 0.0, // y
    0.0f, .5f, 0.0f,}
```



Para enviar los vértices a la pipeline de gráficos empleamos los **Vertex Array Object VAO** que es donde se almacena toda la información relativa al objeto a dibujar, vértices y atributos nuestro caso solo vértices.

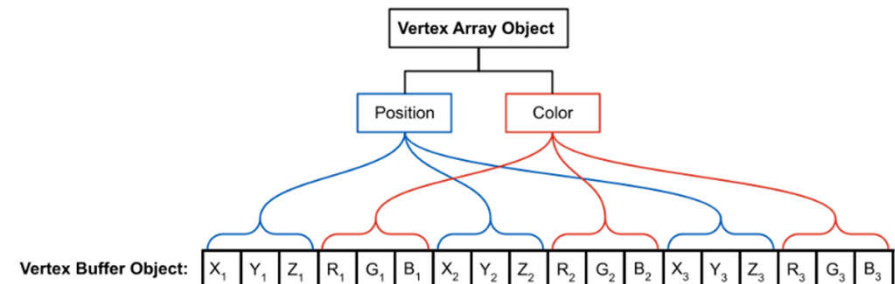
$VAO = VBO(0) + VBO(1) + \dots + VBO(n)$

```
unsigned int VAO; //ID del VAO
glGenVertexArrays(1, &VAO);
glBindVertexArray(VAO);
```

Los atributos (vértices, colores, normales) son añadidos al VAO mediante **Vertex Buffer Object VBO**.

```
unsigned int VBO; ; //ID del VBO
glGenBuffers(1, &VBO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);

glGenBuffers(1, &EBO);
glBindBuffer(GL_ARRAY_BUFFER, EBO);
```



```

void medianteArray() {
// set up vertex data (and buffer(s)) and configure vertex attributes
// -----
float vertices[] = {
0.0f, 0.0f, 0.0f, // x
.5f, 0.0f, .0f,
0.0f, 0.0f, 0.0, // y
0.0f, .5f, 0.0f,
0.0f, 0.0f, 0.0f, // z
0.0f, 0.0f, .5f,
};

```

$$VAO = VBO(0) + VBO(1) + \dots + VBO(n)$$

```

glGenVertexArrays(1, &VAO); // GENERO EL VAO y le hago el bind para que este activo
glBindVertexArray(VAO);
glGenBuffers(1, &VBO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

```

```

// Desvinculo VBO
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0); //Desvinculo VAO

```

```

}
glUseProgram(shaderProgram); // usamos este shader
glBindVertexArray(VAO); // hacemos un bind al VAO
glDrawArrays(GL_LINES, 0, 6); // Para dibujar con array
glBindVertexArray(0); // no need to unbind it every time

```

Como lo usamos.

3.3.

- Tenemos nuestro VAO.
- Tenemos nuestros shader.

```
shaderProgram = setShaders("shader.vert", "shader.frag");  
medianteArray(); // Genero el VAO
```

```
while (!glfwWindowShouldClose(window))  
{
```

```
    //limpieza de los buffers
```

```
    glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
```

```
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    glUseProgram(shaderProgram); // usamos el shader
```

```
    glBindVertexArray(VAO); //hacemos un bind al array
```

```
    glDrawArrays(GL_LINES, 0, 6); //Para dibujar con array
```

```
    glBindVertexArray(0); // no need to unbind it every time
```

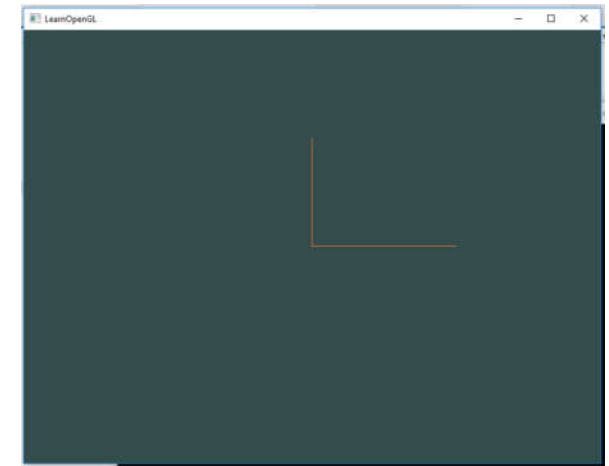
```
    // glfw: swap buffers and poll IO events (keys pressed/released, mouse moved etc.)
```

```
    // -----
```

```
    glfwSwapBuffers(window);
```

```
    glfwPollEvents();
```

```
}
```



```
void glDrawArrays( GLenum mode, GLint first, GLsizei count);
```

Parameters

Mode.- Specifies what kind of primitives to render.

First.-Specifies the starting index in the enabled arrays.

Count.-Specifies the number of indices to be rendered

Como gestiono los vértices. Vertex.shader

- **Procesamiento de vértices en el pipeline gráfico**

Cuando los vértices son enviados al pipeline gráfico de OpenGL, su gestión comienza con el vertex shader, cuyo propósito principal es calcular la posición final de cada vértice en el espacio de pantalla.

- **Entrada de datos al Vertex Shader**

Los datos llegan al shader a través de un **VAO (Vertex Array Object)**, que contiene la información sobre cómo están organizados y almacenados los datos en memoria.

- **Estructura de datos en el VAO**

El diseño o *layout* del VAO define cómo se distribuyen los atributos de cada vértice. Por ejemplo, en el *offset* 0 del VAO suele encontrarse un vector de tres componentes (vec3) que representa la posición del vértice. Este se asigna a una variable de entrada en el shader, comúnmente llamada *aPos*.

- **Cálculo de la posición final**

El vertex shader utiliza una **variable predefinida** llamada `gl_Position` para definir la posición transformada del vértice. Esta variable es obligatoria y debe ser asignada dentro del shader. Su valor debe ser un **vec4**, es decir, un vector de cuatro componentes: las tres coordenadas espaciales (x, y, z) más una cuarta componente w, que normalmente se establece en 1.0.

```
#version 330 core
layout (location = 0) in vec3 aPos;
void main() {
    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0); }
```


Fragment.shader

- Este programa es el que se encarga de calcular el color de cada pixel. En nuestro caso puesto que los vertices (ejes) no tiene color ni ninguna particularidad se la datemos.
- Tiene una variable predefinida que es `gl_FragColor` que define el color de cada fragmento. En nuestro caso el fragment.shader tendrá la forma.

```
#version 330 core
out vec4 FragColor;

void main(){
FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);
}
```

```
#version 330 core

void main()
{
gl_FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);
}
```

- El color de los ejes en RGBA será `vec4(1.0f, 0.5f, 0.2f, 1.0f);`

Como mando los vértices indexados

3.3.

- Si lo dibujo mediante array `glDrawArrays(GL_LINES, 0, 6);`

```
float vertices[] = {  
0.0f, 0.0f, 0.0f, // 0  
.5f, 0.0f, 0.0f, //x  
0.0f, .5f, 0.0f, // y  
0.0f, 0.0f, .5f // z  };  
unsigned int indices[] = {0, 1, 0, 2, 0, 3 };
```

```
glGenVertexArrays(1, &VAO);  
glGenBuffers(1, &VBO);  
glGenBuffers(1, &EBO);  
// bind the Vertex Array Object Para saber sobre que array de vertices se trabaja..  
glBindVertexArray(VAO);  
//A continuacion los atributos  
glBindBuffer(GL_ARRAY_BUFFER, VBO); //hago el bind buffer al vertexArray de trabajo  
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW); //indico el formato
```

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO); // indica index es otro buffer  
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);
```

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);  
glEnableVertexAttribArray(0);
```

```
glBindVertexArray(0);  
}
```

```
glUseProgram(shaderProgram); // leemos el shader  
glBindVertexArray(VAO); //hacemos un bind al array  
glDrawElements(GL_LINES, 6, GL_UNSIGNED_INT, 0);  
glBindVertexArray(0); // no need to unbind it every time
```

Como lo usamos.

3.3.

- Tenemos nuestro VAO.
- Tenemos nuestros shader.

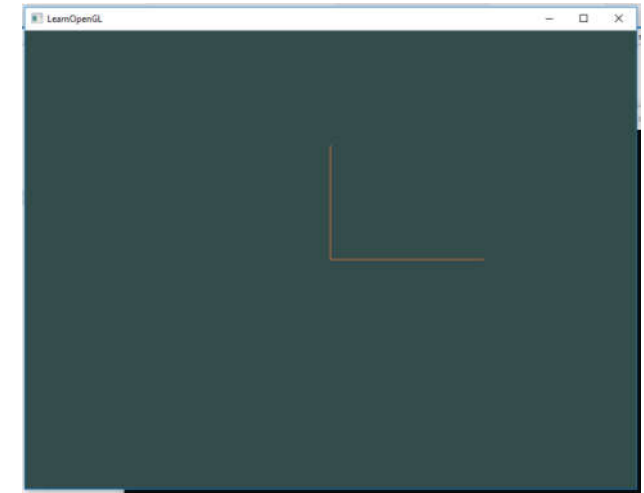
```
shaderProgram = setShaders("shader.vert", "shader.frag");  
medianteArray(); // Genero el VAO
```

```
while (!glfwWindowShouldClose(window))  
{  
    //limpieza de los buffers  
    glClearColor(0.2f, 0.3f, 0.3f, 1.0f);  
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    glUseProgram(shaderProgram); // leemos el shader  
    glBindVertexArray(VAO); //hacemos un bind al array  
    glDrawArrays(GL_LINES, 0, 6); //Para dibujar con array  
    glBindVertexArray(0); // no need to unbind it every time
```

```
    // glfw: swap buffers and poll IO events (keys pressed/released, mouse moved etc.)  
    // -----
```

```
    glfwSwapBuffers(window);  
    glfwPollEvents();  
}
```



```
void glDrawArrays(GLenum mode, GLint first, GLsizei count);
```

Parameters mode GL_POINTS, GL_LINE_STRIP,.....
First. Specifies the starting index in the enabled arrays.
Count.-
Specifies the number of indices to be rendered.

Con más atributos

3.3.

```
float vertices[] = {  
//Vertices          //Colores  
0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f,  // 0  
.5f, 0.0f, 0.0f,    1.0f, 0.0f, 0.0f,  //x  
0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f,  // 0  
0.0f, .5f, 0.0f,    0.0f, 1.0f, 0.0f,  // y  
0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f,  // 0  
0.0f, .5f, 0.0f,    0.0f, 0.0f, 1.0f,  // z  
0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f,  // 0  
.5f , .5f, 0.5f,    1.0f, 1.0f, 1.0f  // 1,1,1 bueno realmente la mitad  
};
```

```
glGenVertexArrays(1, &VAO);  
glGenBuffers(1, &VBO);  
glBindVertexArray(VAO);  
glBindBuffer(GL_ARRAY_BUFFER, VBO);  
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

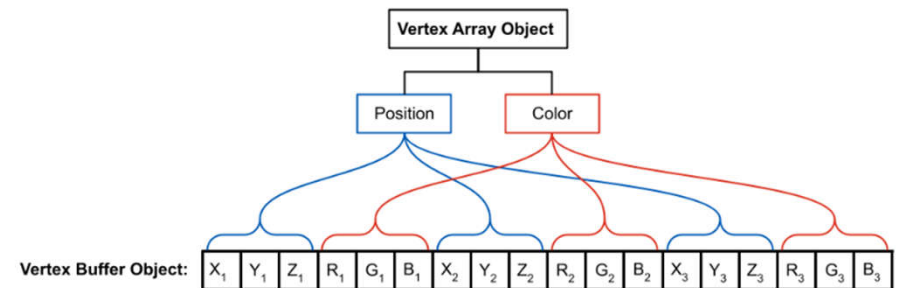
```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0); // position  
attribute
```

```
glEnableVertexAttribArray(0);
```

```
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float)));  
position Color
```

```
glEnableVertexAttribArray(1);  
glBindBuffer(GL_ARRAY_BUFFER, 0);  
glBindVertexArray(0);  
glDeleteBuffers(1, &VBO);
```

```
}
```



$$VAO = VBO(0) + VBO(1) + \dots + VBO(n)$$

```
// dibujo los ejes
```

```
glUseProgram(shaderProgram);  
glBindVertexArray(VAO);  
glDrawArrays(GL_LINES, 0, 8);
```

Shader con Color dos atributos.

3.3.

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0); // position attribute  
glEnableVertexAttribArray(0);
```

```
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float))); //  
position Color  
glEnableVertexAttribArray(1);
```

- En este caso nuestro VAO nos enviara un array con dos atributos los vértices y los colores de los mismos como los gestiona.

```
#version 330 core  
layout (location = 0) in vec3 aPos;  
layout (location = 1) in vec3 aColor;  
out vec3 ourColor;  
void main()  
{ gl_Position = vec4(aPos, 1.0);  
  ourColor = aColor; }
```

```
#version 330 core  
in vec3 ourColor;  
void main()  
{  
  gl_FragColor = vec4 (ourColor,1.0f);  
}
```

• Localización.

• Out para enviar al fragment.

• Igualar el color de salida con el de entrada.

• In color desde el vertex.sh.

• Igual al color para el color buffer

Con más atributos

3.3.

```
float vertices[] = {  
    //Vertices      //Colores  
    0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f, // 0  
    .5f, 0.0f, 0.0f,    1.0f, 0.0f, 0.0f, //x  
    0.0f, .5f, 0.0f,    0.0f, 1.0f, 0.0f, // y  
    0.0f, .5f, 0.0f,    0.0f, 0.0f, 1.0f, // z  
    .5f, .5f, 0.5f,    1.0f, 1.0f, 1.0f // 1,1,1};
```

```
unsigned int indices[] = {0, 2, 0, 3, 0, 4};
```

```
glGenVertexArrays(1, &VA0);  
glGenBuffers(1, &VBO);  
glGenBuffers(1, &EBO);  
// bind the VAO first, then bind and set vertex buffer(s), and then configure vertex attributes(s).  
glBindVertexArray(VA0);
```

```
glBindBuffer(GL_ARRAY_BUFFER, VBO);  
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

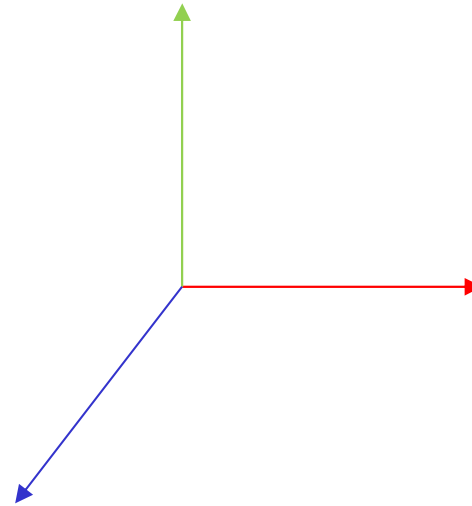
```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0); // position attribute  
glEnableVertexAttribArray(0);
```

```
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float))); // pos  
glEnableVertexAttribArray(1);
```

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);  
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);
```

```
glDeleteBuffers(1, &VBO);  
glDeleteBuffers(1, &EBO);  
}
```

```
glUseProgram(shaderProgram);  
glBindVertexArray(VA0);  
glDrawElements(GL_LINES, 8, GL_UNSIGNED_INT, 0);  
glBindVertexArray(0); // no need to unbind it every time
```



+ Sobre Shaders

Embedded en nuestro código

```
// Vertex Shader
const char* vertexShaderSource = "#version
330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"void main()\n"
"{\n"
"    gl_Position = vec4(aPos, 1.0);\n"
"}\0";

// Fragment Shader
const char* fragmentShaderSource = "#version
330 core\n"
"out vec4 FragColor;\n"
"void main()\n"
"{\n"
"    FragColor = vec4(1.0, 0.0, 0.0, 1.0);\n"
"}\n\0";
```

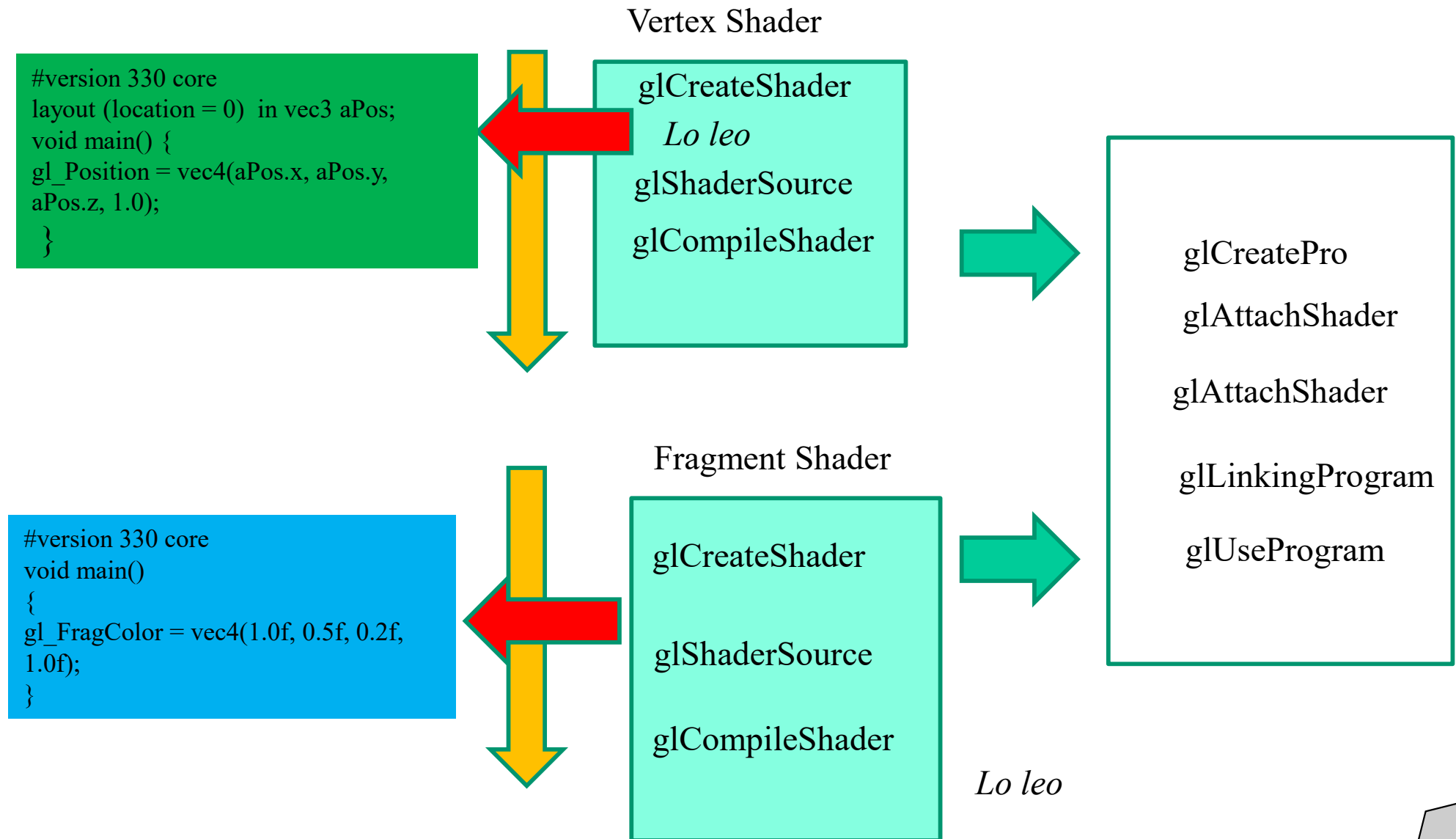
En Nuestro Disco

```
#version 330 core
layout (location = 0) in vec3 aPos;
void main() {
    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
}
```

```
#version 330 core
void main()
{
    gl_FragColor = vec4(1.0f, 0.0f, 0.0f, 1.0f);
}
```

Programa Vertex y Shader. GLSL es

Tenemos nuestro shaders. Como los metemos en nuestro programa



Crear un program shader

```
unsigned int GeneroShaders() {
```

```
// Build and compile the shader program
```

```
unsigned int vertexShader, fragmentShader, shaderProgram;
```

```
vertexShader = glCreateShader(GL_VERTEX_SHADER);
```

```
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
```

```
glCompileShader(vertexShader);
```

```
fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
```

```
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
```

```
glCompileShader(fragmentShader);
```

```
shaderProgram = glCreateProgram();
```

```
glAttachShader(shaderProgram, vertexShader);
```

```
glAttachShader(shaderProgram, fragmentShader);
```

```
glLinkProgram(shaderProgram);
```

```
glDeleteShader(vertexShader);
```

```
glDeleteShader(fragmentShader);
```

```
return shaderProgram; }
```

```
// Vertex Shader
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"void main()\n"
"{\n"
"    gl_Position = vec4(aPos, 1.0);\n"
"}\n";

// Fragment Shader
const char* fragmentShaderSource = "#version 330 core\n"
"out vec4 FragColor;\n"
"void main()\n"
"{\n"
"    FragColor = vec4(1.0, 0.0, 0.0, 1.0);\n"
"}\n";
```

Progama+vertx.frag

3.3.

```
GLuint setShaders(const char *nVertx, const char *nFrag) {  
    char *ficheroVs = NULL;  
    char *ficheroFs = NULL;  
    const char *codigoVs = NULL;  
    const char *codigoFs = NULL;  
  
    vertexShader = glCreateShader(GL_VERTEX_SHADER); //Creo el vertexShader y  
    el FragmentShader  
    fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);  
  
    ficheroVs = readFile(nVertx); //Leo el codigo del fichero  
    ficheroFs = readFile(nFrag);  
  
    codigoVs = ficheroVs; //Lo igual al puntero para cargarlos  
    codigoFs = ficheroFs;  
  
    glShaderSource(vertexShader, 1, &codigoVs, NULL); //Los cargo  
    glShaderSource(fragmentShader, 1, &codigoFs, NULL);  
    free(ficheroVs); free(ficheroFs); //Libero los ficheros  
  
    glCompileShader(vertexShader); // Compilo vertex y Fragment  
    glCompileShader(fragmentShader);  
  
    printShaderInfoLog(vertexShader); //Miro si hay algun error  
    printShaderInfoLog(fragmentShader);  
}
```

```
//Creo el programa asociado  
progShader = glCreateProgram();  
  
// Le attacheo shaders al programa  
glAttachShader(progShader, vertexShader);  
glAttachShader(progShader, fragmentShader);  
  
// Lo linko  
glLinkProgram(progShader);  
// A ver si hay errores  
printProgramInfoLog(progShader);  
  
//Lo uso  
glUseProgram(progShader);  
return (progShader);  
}
```

```
#version 330 core  
layout (location = 0) in vec3 aPos;  
void main() {  
    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);  
}
```

```
#version 330 core  
void main()  
{  
    gl_FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);  
}
```